

Variables and Literals

Understanding data storage, static typing, and value representation in Java

In the previous tutorial, you wrote your first Java program. Now let's understand something fundamental: **How Java stores data**. That's where **variables** and **literals** come in.

1. Java Variables

A **variable** is a named memory location used to store data.

Every variable in Java must have:

- A **Data Type** (defines what kind of data can be stored)
- A **Unique Name** (identifier used to access the memory location)
- A **Value** (optional at declaration)

Creating Variables in Java

Let's create a variable that stores the number of problems solved:

```
int problemsSolved = 120;
```

Explanation:

- `int` → Data type
- `problemsSolved` → Variable name
- `120` → Value stored

This means we are storing the value `120` inside a memory location named `problemsSolved`.

Declaration and Assignment

You can declare and assign separately:

```
double completionRate;  
completionRate = 87.5;
```

Or do both together in a single statement:

```
boolean isPremiumMember = true;
```

Both approaches are completely valid.

Java is Statically Typed

Java requires that every variable be declared before it is used.

Valid Declaration **VALID**

```
int totalContests = 15;
```

Invalid Declaration **INVALID**

```
totalContests = 15; // ❌ Type missing
```

Java must know the data type first before allocating memory.

Changing Variable Values

The value stored inside a variable can change during program execution.

```
int rank = 450;

// After next contest
rank = 320;
```

The memory location stays the same, but the value updates. That's why it is called a *variable*.

Important Rule: Type Cannot Change

Once declared, a variable cannot change its data type or be redeclared with a different type in the same scope.

```
int streak = 30;
double streak = 30.5; // ❌ Not allowed - Redefinition error
```

Naming Rules for Variables

Follow these rules when naming variables:

- Can contain letters, digits, `_` (underscore), and `$` (dollar sign).
- **Cannot** start with a digit.
- **Cannot** use Java reserved keywords (e.g., `class`, `int`, `static`).
- Names are **case-sensitive**.

```
int score;
int Score; // Different variable from 'score'
```

2. Java Literals

A **literal** is the actual fixed value assigned to a variable.

```
int activeUsers = 5000;
```

Here: `activeUsers` is the variable, and `5000` is the literal. Java supports multiple types of literals.

1 Integer Literals

Integer literals represent whole numbers. Java supports 4 number formats:

- **Decimal (Base 10):** Standard numbers.

```
int enrolledStudents = 250;
```

- **Binary (Base 2):** Uses prefix `0b` or `0B`.

```
int accessMask = 0b10101;
```

- **Octal (Base 8):** Uses leading `0`.

```
int filePermission = 075;
```

- **Hexadecimal (Base 16):** Uses prefix `0x` or `0X`.

```
int themeColor = 0xFF;
```

Integer literals are used with integral types: `byte`, `short`, `int`, and `long`.

2 Floating-Point Literals

Floating-point literals represent decimal numbers.

```
double averageScore = 78.6;  
float discountPercent = 15.5F;
```

By default: Fractional numbers are treated as `double`. To store them in a `float`, you must append an `F` or `f` suffix.

Scientific Notation:

```
double serverLoad = 2.5e3; // 2.5 × 103 = 2500.0
```

This format is useful in data analytics, science, and performance calculations.

3 Character Literals

Character literals represent a single character enclosed in **single quotes**.

```
char grade = 'A';
```

You can also use escape sequences for special characters:

```
char newLine = '  
'; // Newline  
char tabSpace = '    '; // Tab space
```

4 String Literals

A string literal is a sequence of characters inside **double quotes**.

```
String platformName = "CodeHelp ONE";  
String courseTrack = "DSA + Core + Aptitude";
```

5 Boolean Literals

Boolean literals represent logical truth values. They can only be `true` or `false`.

```
boolean isLoggedIn = true;  
boolean hasCompletedSheet = false;
```

💡 Why Variables & Literals Matter for Placements

In technical interviews and coding rounds, companies test:

- Variable declaration clarity and scoping rules
- Correct data type usage and memory efficiency
- Memory understanding (stack vs heap assignment)
- Type mismatch errors and casting nuances
- Edge case handling with boundary values

Most beginner mistakes happen due to a poor understanding of data types, literal formats, and implicit type assumptions. Strong fundamentals here prevent bugs in DSA and system programming.

3. Quick Summary

Concept	Meaning & Description
Variable	Named memory location used to store dynamic data
Literal	Fixed value directly assigned to a variable
Static Typing	Data type must be declared explicitly before variable use
Integer Literal	Whole numbers formatted in Decimal, Binary (0b), Octal (0), or Hex (0x)
Floating Literal	Decimal or exponential numbers (defaults to double unless 'F' appended)
Character Literal	Single character enclosed in single quotes or escape sequences
String Literal	Text/sequence of characters enclosed in double quotes
Boolean Literal	Logical values limited strictly to <code>true</code> or <code>false</code>